ORACLE CLOUD INFRASTRUCTURE

OCI Cloud Lab — Technical Deployment Report

Load-Balanced Private Application with File Storage

Ejada Egypt Summer Internship Program 2026
Cloud Build Track — Week 2, Lab 2

Field	Detail
Prepared by	Andrew Hany
Lab	Week 2 — Lab 2
Virtual Cloud Network	ejada-w2-dev-vcn (10.0.0.0/16)
Region	me-jeddah-1 (Saudi Arabia West — Jeddah)
Tenancy	ociejada — identity domain Ejada-interim-program
Compartment	intern-04-andrew-hany-cmp
Deployment Date	August 12, 2026 (UTC)
Document Type	Technical Deployment & Verification Report
Provisioning Method	Step 1: OCI Console and OCI CLI — Step 2: Terraform

Console and CLI deployment, with the equivalent Terraform implementation in Section 12.

1. Project Overview & Objectives

This report documents the deployment of a two-tier, load-balanced web application on Oracle Cloud Infrastructure, completed as part of the Ejada Egypt Summer Internship Program 2026 — Cloud Build Track. The environment was provisioned through the OCI Console and the OCI CLI, and is rebuilt in full using Terraform as Step 2 of the same lab.

The defining characteristic of the architecture is that the application server holds no public IP address and is not addressable from the internet. All client traffic arrives at a public Application Load Balancer in a public subnet, which forwards it to a private compute instance. The files that the application serves are not stored on the instance at all; they reside on an OCI File Storage file system mounted over NFS, so the compute layer is disposable and the data is not.

Objectives

- Design a Virtual Cloud Network with a public tier for internet-facing traffic and a private tier for the workload.
- Configure Internet, NAT and Service gateways with matching route tables so the private subnet can reach outward but never be reached inward.
- Control traffic using Network Security Groups that reference one another, rather than broad subnet CIDR ranges.
- Provision an OCI File Storage file system, mount target and export, and mount the export on the instance over NFS.
- Launch an Oracle Linux 9 compute instance in the private subnet with no public IP address.
- Publish the application through a public Application Load Balancer with an HTTP health check.
- Verify the environment end to end, then reproduce it entirely in Terraform.

Scope

This report covers the network, storage, compute and load balancing configuration evidenced by the OCI Console screenshots and CLI output captured during the deployment. Identity and Access Management policy design is out of scope, although two IAM constraints encountered during the lab are documented in Section 9, since they materially changed the implementation approach.

2. Prerequisites and Address Plan

Item	Value / Status
OCI Tenancy	ociejada
Identity Domain	Ejada-interim-program
Region	me-jeddah-1
Working Compartment	intern-04-andrew-hany-cmp
Console Access	OCI Web Console (browser-based)
CLI Access	OCI Cloud Shell (pre-authenticated OCI CLI)
Target OS Image	Oracle Linux 9, image build 2026.07.20-1
Compute Shape	VM.Standard.E5.Flex — 1 OCPU, 12 GB memory

Address plan

The address plan was fixed before any resource was created. Every subnet is derived from a single VCN CIDR, which keeps the design coherent and makes the Terraform implementation in Step 2 able to compute both subnets from one variable.

Item	CIDR	Purpose
VCN	10.0.0.0/16	Entire network — 65,536 addresses
Public subnet	10.0.1.0/24	Application Load Balancer
Private subnet	10.0.2.0/24	Compute instance and File Storage mount target

Note A File Storage mount target consumes three IP addresses in its subnet, so a /30 or smaller subnet cannot host one. The /24 used here leaves ample headroom.

3. Virtual Cloud Network

The network foundation is a single VCN, ejada-w2-dev-vcn, created with the standard Create VCN action rather than the VCN Wizard. The wizard would have auto-provisioned its own subnets, gateways and route rules; building each component explicitly makes the public/private split visible in the configuration and keeps the manual build aligned one-to-one with the Terraform written for Step 2.

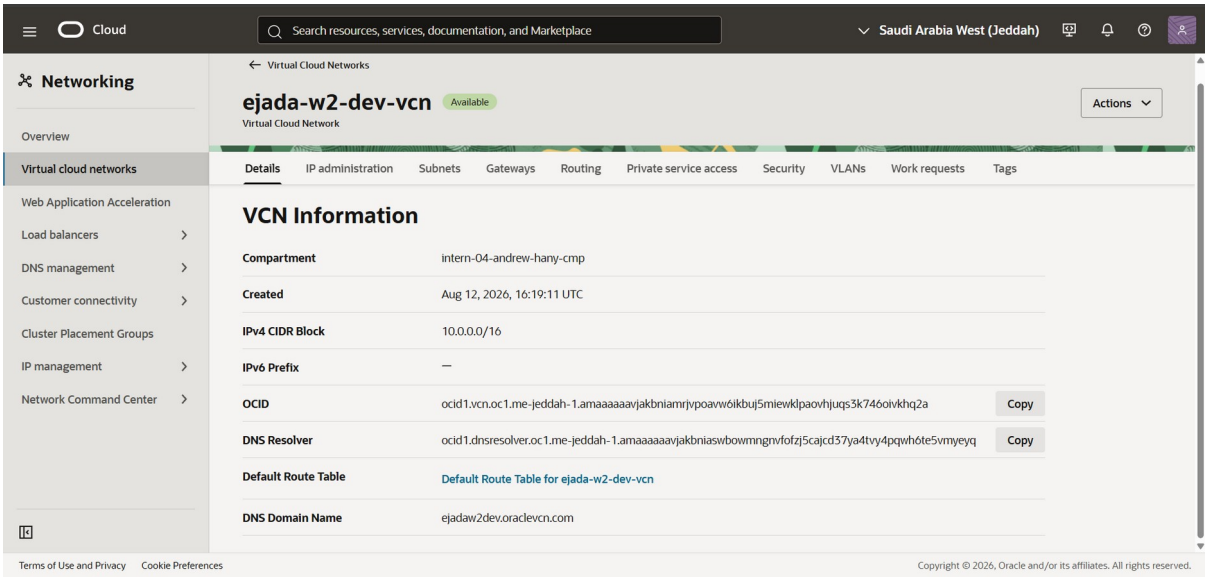

Figure 1 — ejada-w2-dev-vcn details: CIDR 10.0.0.0/16, compartment intern-04-andrew-hany-cmp, state Available.

Attribute	Value
Name	ejada-w2-dev-vcn
IPv4 CIDR Block	10.0.0.0/16
IPv6 Prefix	Not configured
DNS Domain Name	ejadaw2dev.oraclevcn.com
Default Route Table	Default Route Table for ejada-w2-dev-vcn (left unused)
State	Available
Created	Aug 12, 2026, 16:19:11 UTC

The VCN ships with a Default Route Table, Default Security List and Default DHCP Options. All three were left empty and unattached. Purpose-built route tables and security lists were created instead and attached to each subnet explicitly, so that the distinction between the public and private tiers is expressed in the configuration rather than hidden behind shared defaults.

4. Gateways and Routing

Three gateways were attached to the VCN, one for each distinct traffic pattern the design requires.

Gateway	Name	Role
Internet Gateway	ejada-w2-dev-igw	Bidirectional internet access for the public subnet
NAT Gateway	ejada-w2-dev-natgw	Outbound-only internet access for the private subnet
Service Gateway	ejada-w2-dev-sgw	Access to OCI services over the Oracle backbone

The NAT Gateway is not optional in this design. "Private" means unreachable from the internet, not unable to reach out: without a NAT Gateway the instance could not install Apache or the NFS client from

the Oracle Linux repositories, and the Oracle Cloud Agent could not report its status to the OCI control plane. The asymmetry it provides — outbound connections permitted, inbound connections impossible — is the practical definition of a private subnet in OCI.

Route tables

Two route tables were created and the default was left with zero rules. The private table carries the clearest single expression of the design intent.

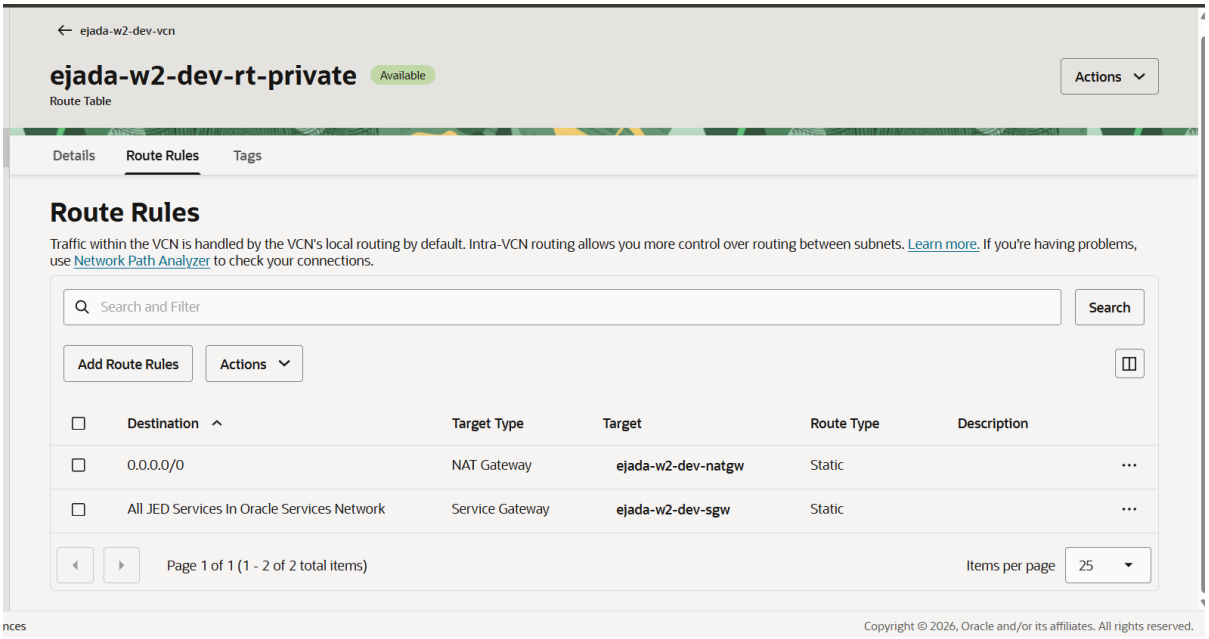

Figure 2 — ejada-w2-dev-rt-private: default route via the NAT Gateway, and the Oracle Services Network service CIDR label via the Service Gateway.

Destination	Target Type	Target
0.0.0.0/0	Internet Gateway	ejada-w2-dev-igw (public table)
0.0.0.0/0	NAT Gateway	ejada-w2-dev-natgw (private table)
All JED Services In OSN	Service Gateway	ejada-w2-dev-sgw (private table)

The Service Gateway rule is more specific than 0.0.0.0/0, so OCI's longest-prefix-match routing selects it for traffic bound to an OCI service, keeping that traffic on the Oracle backbone rather than sending it out through NAT.

> **Note** Neither table contains a rule for 10.0.0.0/16. Routing between subnets inside a VCN is implicit and cannot be removed. This matters later: the load balancer in 10.0.1.0/24 reaches the instance in 10.0.2.0/24 with no route rule involved — that path is governed purely by security rules.

5. Security Lists and Network Security Groups

Traffic control is split deliberately between two mechanisms. Two security lists were created, one per subnet, and both were kept intentionally thin — they carry only an outbound allow rule and an ICMP path-MTU rule. Every application-level rule lives in one of three Network Security Groups instead.

	Security List	Network Security Group
Attached to	A subnet — applies to every VNIC in it	Specific VNICs
Rule sources	CIDR or service only	CIDR, service, or another NSG
Membership	Implicit (be in the subnet)	Explicit (attach the NSG)

OCI evaluates the two as a union: a packet is permitted if either the subnet's security list or an attached NSG allows it, and neither can override the other to deny. That union behaviour is what makes this split safe, and it is why the default security list was deliberately not attached to either subnet — its permissive rules, including SSH from 0.0.0.0/0, could not have been overridden by anything else in the design.

Because an NSG rule can name another NSG as its source, a rule can express "only the load balancer may reach the application" rather than "anything in 10.0.1.0/24 may reach the application". If another resource is later placed in the public subnet, it does not silently inherit access to the private tier.

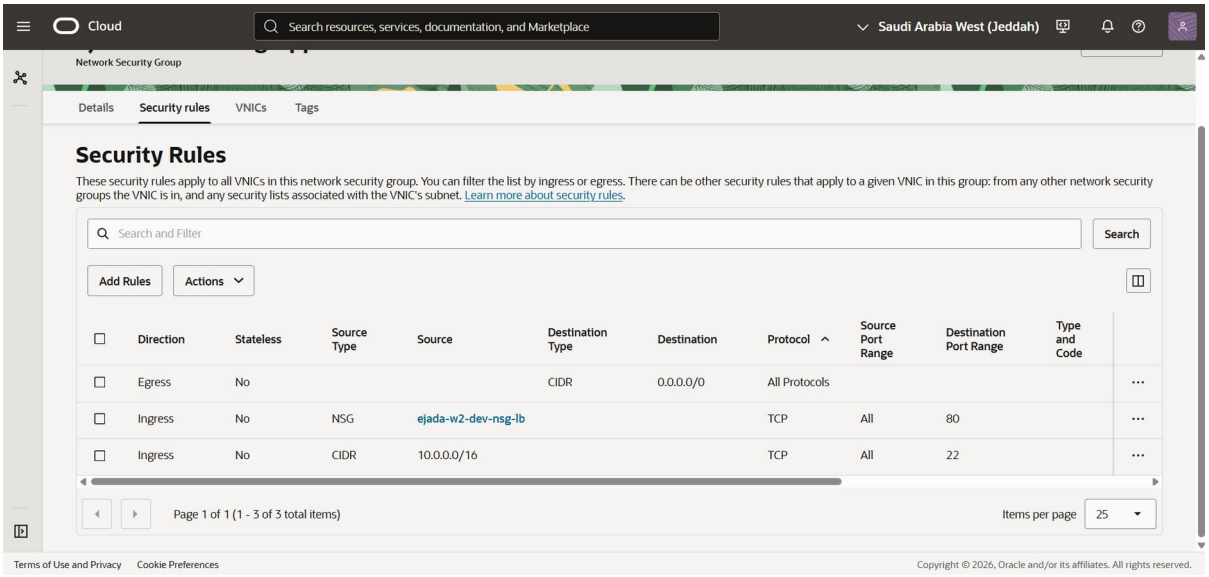

Figure 3 — ejada-w2-dev-nsg-app: note the ingress rule whose Source Type is NSG and whose Source is ejada-w2-dev-nsg-lb, rather than a CIDR block.

Rule summary

NSG	Direction	Source / Destination	Protocol	Port
nsg-lb	Ingress	0.0.0.0/0 (CIDR)	TCP	80
nsg-lb	Egress	nsg-app (NSG)	TCP	80
nsg-app	Ingress	nsg-lb (NSG)	TCP	80
nsg-app	Ingress	10.0.0.0/16 (CIDR)	TCP	22
nsg-app	Egress	0.0.0.0/0 (CIDR)	All	—
nsg-fss	Ingress	nsg-app (NSG)	TCP	111
nsg-fss	Ingress	nsg-app (NSG)	TCP	2048–2050
nsg-fss	Ingress	nsg-app (NSG)	UDP	111

NSG	Direction	Source / Destination	Protocol	Port
nsg-fss	Ingress	nsg-app (NSG)	UDP	2048
nsg-fss	Egress	nsg-app (NSG)	All	—

All rules are stateful, so replies are permitted automatically and no matching rule is required in the opposite direction. The six NFS ports are mandatory: 111 is the RPC portmapper and 2048–2050 carry mountd, nfsd and statd. If any one of them is missing, the mount command does not return an error — it hangs until it times out, because the RPC negotiation never completes.

The port 22 rule on nsg-app is sourced from the VCN CIDR rather than a public address because the OCI Bastion service does not connect from an administrator workstation; it provisions a private endpoint inside the VCN and brokers the session from there.

6. Subnets

Two regional subnets were carved out of the VCN CIDR, one per tier. Creating them is the point at which the route tables and security lists stop being inert definitions and begin governing traffic — a subnet is where a route table, a security list and an address range are bound together.

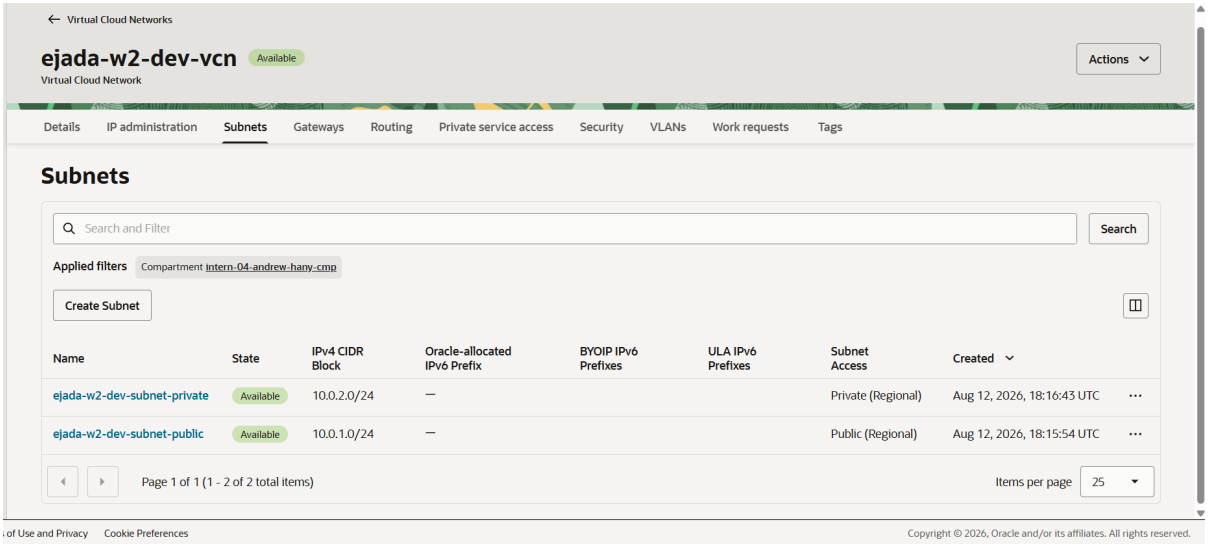

Figure 4 — Both subnets Available: 10.0.1.0/24 Public (Regional) and 10.0.2.0/24 Private (Regional).

Attribute	Public subnet	Private subnet
Name	ejada-w2-dev-subnet-public	ejada-w2-dev-subnet-private
CIDR Block	10.0.1.0/24	10.0.2.0/24
Subnet Type	Regional	Regional
Subnet Access	Public	Private
Route Table	ejada-w2-dev-rt-public	ejada-w2-dev-rt-private
Security List	ejada-w2-dev-sl-public	ejada-w2-dev-sl-private
Contains	Application Load Balancer	Instance, FSS mount target

Selecting Private Subnet sets the subnet's prohibit_public_ip_on_vnic attribute. This is an enforced constraint rather than a default: the Console will not offer to assign a public IP to an instance launched into this subnet, and an API call requesting one is rejected. The application instance therefore has no public IP not because the option was left unticked at launch, but because the subnet makes a public IP impossible — a guarantee that remains true for anything else placed there later.

7. File Storage

OCI File Storage provides the shared, NFS-mounted storage that holds the application's files, satisfying the requirement that the application files be stored on File Storage and mounted to the instance. Three distinct objects are involved, and the distinction matters.

Object	What it is
File system	The storage itself — has an OCID and an availability domain, but no IP address and no network presence
Mount target	The NFS endpoint — lives in a subnet, holds a private IP, and is what clients connect to
Export	The binding between the two, published at a path, carrying the access rules

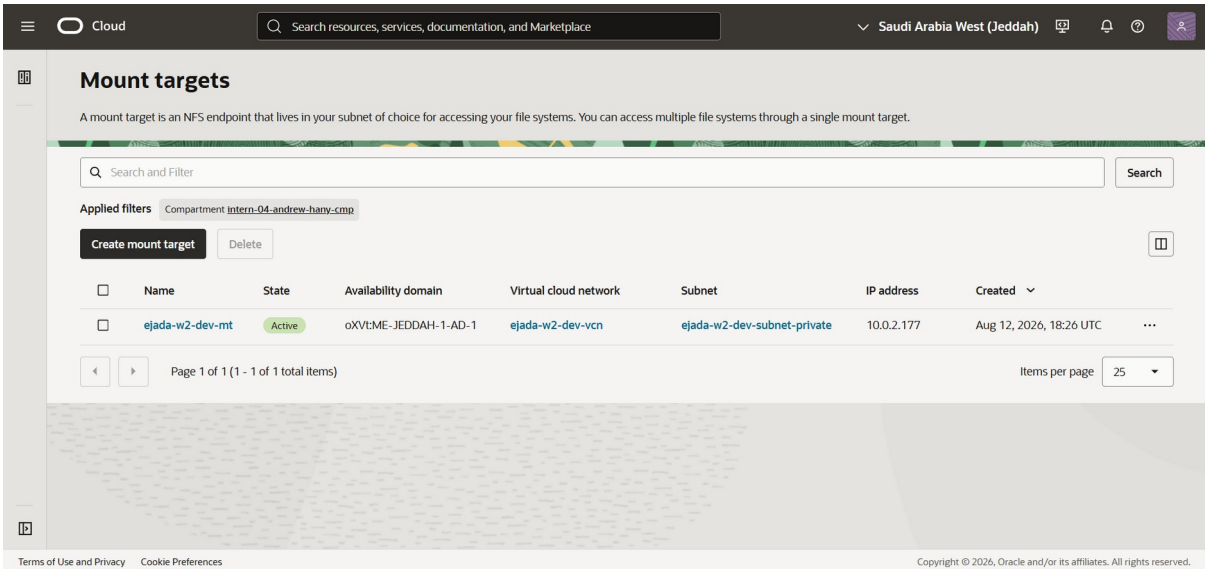

Figure 5 — ejada-w2-dev-mt: Active, private IP 10.0.2.177, placed in ejada-w2-dev-subnet-private.

Attribute	Value
File system	ejada-w2-dev-fs — Active, 0 B at creation
Availability Domain	oXVt:ME-JEDDAH-1-AD-1
Mount target	ejada-w2-dev-mt
Mount target subnet	ejada-w2-dev-subnet-private
Mount target IP address	10.0.2.177
Export path	/app
Created	Aug 12, 2026, 18:26 UTC

The mount target sits in the private subnet. An NFS endpoint placed in a subnet with a route to the internet gateway would be reachable from a far larger surface than intended, for no benefit — the only client is the application instance, which lives in the same subnet, so the NFS traffic never crosses a subnet boundary. File Storage is billed on data actually written rather than a provisioned size, so no capacity had to be chosen in advance.

NFS client export options

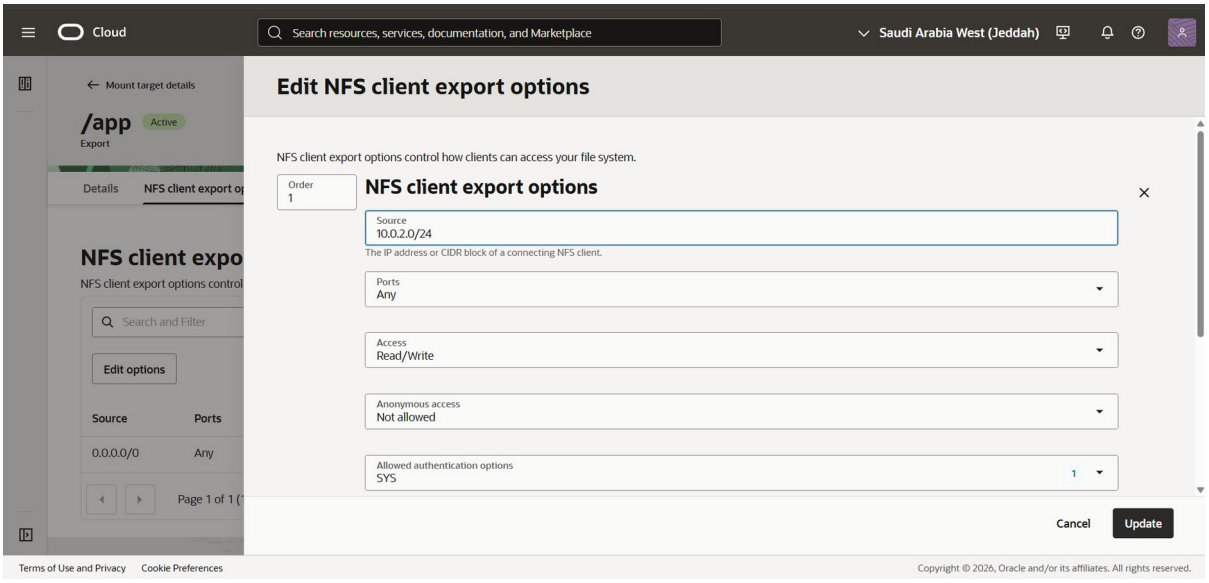

Figure 6 — Export options narrowed to the private subnet CIDR, read/write, with anonymous access disallowed.

Option	Value	Reason
Source	10.0.2.0/24	Only the private subnet may mount
Ports	Any	Clients need not use privileged source ports
Access	Read/Write	The application writes as well as reads
Anonymous access	Not allowed	No identity squashing — root stays root
Authentication	SYS	Standard AUTH_SYS; Kerberos not configured

Two of these deserve defending. The source was narrowed from the default 0.0.0.0/0 to the private subnet CIDR, which combined with the NSG gives two independent layers: the NSG controls which VNICs can reach the endpoint at all, and the export options control which addresses the NFS server will serve. Either would suffice in this topology; both together mean a mistake in one does not expose the data.

Anonymous access "Not allowed" is the Console's presentation of identity_squash = NONE. With squashing enabled, the root user on the client is remapped to an anonymous UID and the chown performed on the mount during bootstrap fails with a permission error that gives no indication that NFS identity mapping is the cause.

8. Compute Instance and Application Deployment

The application server was launched into the private subnet with no public IP address. This is the central requirement of the lab: the workload must be unreachable from the internet directly, and reachable only through the load balancer in the public subnet.

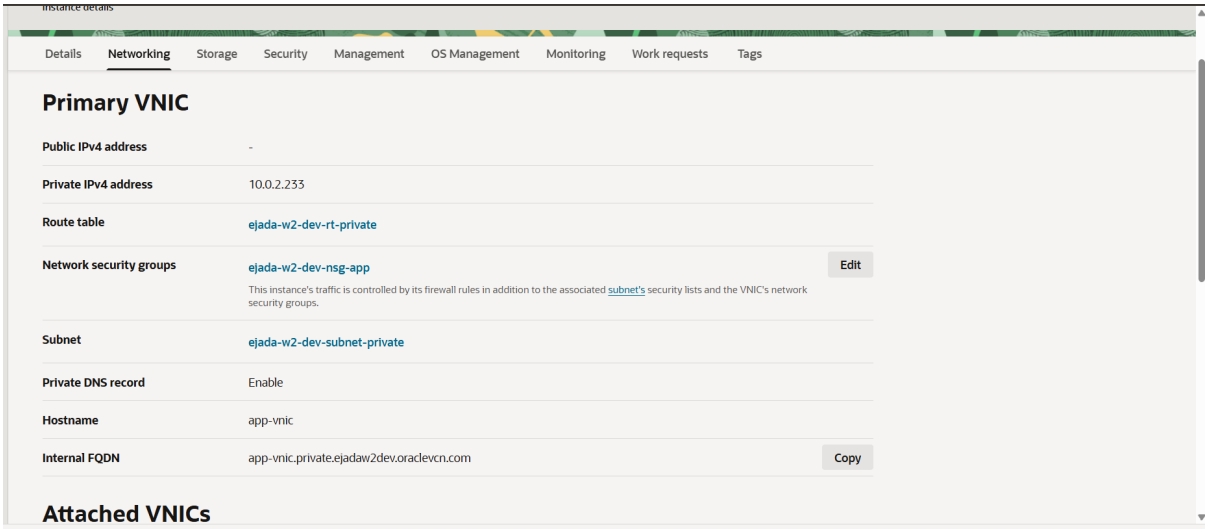

Figure 7 — Primary VNIC of ejada-w2-dev-app: the Public IPv4 address field is empty, the instance holds only a private address, and ejada-w2-dev-nsg-app is attached.

Attribute	Value
Name	ejada-w2-dev-app
Operating System	Oracle Linux 9, build 2026.07.20-1
Shape	VM.Standard.E5.Flex — 1 OCPU, 12 GB, 1 Gbps
Availability / Fault Domain	AD-1 / FD-1
Public IPv4 address	None
Private IPv4 address	10.0.2.185
Subnet	ejada-w2-dev-subnet-private
Route table	ejada-w2-dev-rt-private
Network Security Group	ejada-w2-dev-nsg-app
Internal FQDN	app.private.ejadaw2dev.oraclevcn.com

Note Figure 7 was captured from the first build of this instance, which held 10.0.2.233. The instance was subsequently relaunched with a cloud-init bootstrap (see Section 9, Issue 5) and now holds 10.0.2.185. Every other property shown — absent public IP, private subnet, private route table, attached NSG — is unchanged.

Application bootstrap

Apache and the NFS client were installed by a cloud-init script supplied as instance metadata at launch, rather than through an interactive session. The script mounts the File Storage export at the Apache document root, seeds the application files onto the share if it is empty, and applies the two host-level settings that this configuration requires.

```
dnf install -y nfs-utils httpd
echo "10.0.2.177:/app /var/www/html nfs nosuid,resvport,_netdev,nofail 0 0" >> /etc/fstab
for i in $(seq 1 12); do mount /var/www/html && break; sleep 15; done
```

```
setsebool -P httpd_use_nfs 1            # SELinux: allow Apache to read an NFS mount
firewall-cmd --permanent --add-service=http   # firewalld: open port 80
firewall-cmd --reload
systemctl enable --now httpd
```

Mounting the export directly at /var/www/html means Apache's DocumentRoot is the NFS share. No file the application serves exists on the instance's boot volume, which is what makes the compute layer disposable.

The two host-level settings are the most common causes of failure in this lab and neither produces an obvious error. Oracle Linux 9 blocks Apache from reading an NFS mount under SELinux by default, which presents as HTTP 403 responses that look like a file permissions problem. Port 80 is closed in firewalld on a fresh image, which presents as a load balancer backend stuck in a Critical state while the instance appears entirely healthy from the inside. The mount is retried in a loop because a mount target can take several seconds to answer on a cold boot, and a single failed attempt would otherwise abort the bootstrap.

9. Issues Encountered and Resolutions

Issue 1 — Sign-in rejected at the Cloud Account Name prompt

Symptom: the Oracle Cloud sign-in page returned "That name didn’t work, want to try again?". Cause: the Cloud Account Name field expects the tenancy name, not a user email address. Resolution: signed in with tenancy ociejada, then selected the Ejada-interim-program identity domain rather than Default.

Issue 2 — Compartment page returned an authorization error

Symptom: opening the compartment in Identity & Security returned "Authorization failed or requested resource not found", so the compartment OCID could not be copied from the Console. Cause: the intern user has manage rights on resources inside the compartment but not read on the compartment object in IAM — expected least-privilege behaviour in a shared training tenancy. Resolution: the OCID was recovered from a resource already owned in that compartment, which requires only networking permissions.

```
oci network vcn get --vcn-id <vcn-ocid> \
  --query 'data."compartment-id"' --raw-output
```

Every OCI resource carries its compartment-id, so a compartment OCID can always be recovered from any resource inside it without IAM read access.

Issue 3 — NSG toggle disabled in the Create Instance wizard

Symptom: the "Use network security groups to control traffic" toggle remained greyed out with the hint "Select a VCN before using network security groups", even with the VCN and subnet correctly selected. Impact if unnoticed: the private security list permits only ICMP, so an instance launched with no NSG would have been unreachable on both port 80 and port 22 — the load balancer backend would have reported Critical and no symptom would have pointed at the cause. Resolution: the NSG was attached after launch through Attached VNICs, and in the final build the instance was launched from the CLI with --nsg-ids, which sets the group at creation.

Issue 4 — OCI Bastion could not be created (IAM)

Symptom: the Create Bastion button in the Console spun and returned without feedback. The CLI revealed the underlying error.

```
"code": "NotAuthorizedOrNotFound",
"operation_name": "create_bastion",
"status": 404,
"target_service": "bastion"
```

Diagnosis: listing bastions in the working compartment succeeded and returned an empty result, while listing in the parent compartment and creating in the working compartment both failed. The account therefore holds read on bastion-family but not manage. This is a policy gap rather than a configuration error, and it cannot be worked around from the user side.

Impact: Managed SSH sessions were unavailable, so the private instance could not be reached interactively. A request for "allow group <interns> to manage bastion-family in compartment intern-04-andrew-hany-cmp" was raised with the tenancy administrator.

Issue 5 — Compute Instance Run Command also unavailable, and the resulting approach change

Symptom: with bastion unavailable, the Oracle Cloud Agent Run Command plugin was attempted as an alternative means of executing the installation script. It returned the same authorization failure against the compute_instance_agent service.

Resolution: with no interactive path to the instance, the application was installed through cloud-init instead. Because cloud-init runs only on first boot, the existing instance was terminated and relaunched from the CLI with the bootstrap script supplied as user data. A temporary ingress rule permitting the private subnet CIDR was added to nsg-fss beforehand, because the instance mounts the file system during boot — before the NSG-to-NSG rule can take effect.

```
oci compute instance launch -c $CMP \
  --availability-domain "$AD" --display-name ejada-w2-dev-app \
  --shape VM.Standard.E5.Flex --shape-config '{"ocpus":1,"memoryInGBs":12}' \
  --image-id $IMG --subnet-id $PRIV --assign-public-ip false \
  --nsg-ids "[\"$NSGAPP\"]" --user-data-file ~/ci.sh --wait-for-state RUNNING
```

This approach is arguably closer to production practice than an interactive install: the instance configuration is expressed as code, is reproducible, and requires no inbound administrative access at all. It is also what the Terraform in Step 2 does, using the same script rendered through templatefile().

Observation — Console limitations that Terraform does not share

Two of the frictions above are artefacts of building by hand. An NSG rule cannot reference an NSG that does not yet exist, so the Console build had to be sequenced manually: create nsg-lb with only its ingress rule, create nsg-app, then return to nsg-lb to add the egress rule. Separately, the Create Instance wizard would not attach an NSG at launch. Terraform has neither problem — it infers the dependency graph from resource references and sets the NSG as a field on the VNIC.

```
create_vnic_details {
  subnet_id        = oci_core_subnet.private.id
  assign_public_ip = false
  nsg_ids          = [oci_core_network_security_group.app.id]
}
```

10. Verification Checklist

Verification Item	Status	Evidence
VCN created with CIDR 10.0.0.0/16	PASS	Figure 1
Internet, NAT and Service gateways Available	PASS	Section 4
Public route table: 0.0.0.0/0 to Internet Gateway	PASS	Section 4
Private route table: 0.0.0.0/0 to NAT, OSN to Service GW	PASS	Figure 2
Three NSGs created using NSG-to-NSG rules	PASS	Figure 3
Public subnet 10.0.1.0/24 created	PASS	Figure 4
Private subnet 10.0.2.0/24, public IPs prohibited	PASS	Figure 4
File system and mount target in the private subnet	PASS	Figure 5
Export /app restricted to 10.0.2.0/24, no identity squash	PASS	Figure 6
Instance launched with no public IP address	PASS	Figure 7
Application installed via cloud-init bootstrap	PASS	Section 8
File Storage export mounted at the Apache document root	PENDING	LB health
Load balancer backend set reporting healthy	PENDING	To capture
Application reachable at the load balancer public IP	PENDING	To capture

Note The three pending items are verified by a single observation. Because the instance cannot be accessed interactively, a healthy load balancer backend is the end-to-end proof: it demonstrates simultaneously that cloud-init completed, that the NFS mount succeeded, that Apache is serving from it, and that the SELinux boolean and firewall rule were applied.

11. Architecture Diagram

The diagram below consolidates every component described in this report and the paths between them. It was produced in draw.io; the editable source is included with the submission as architecture.drawio.

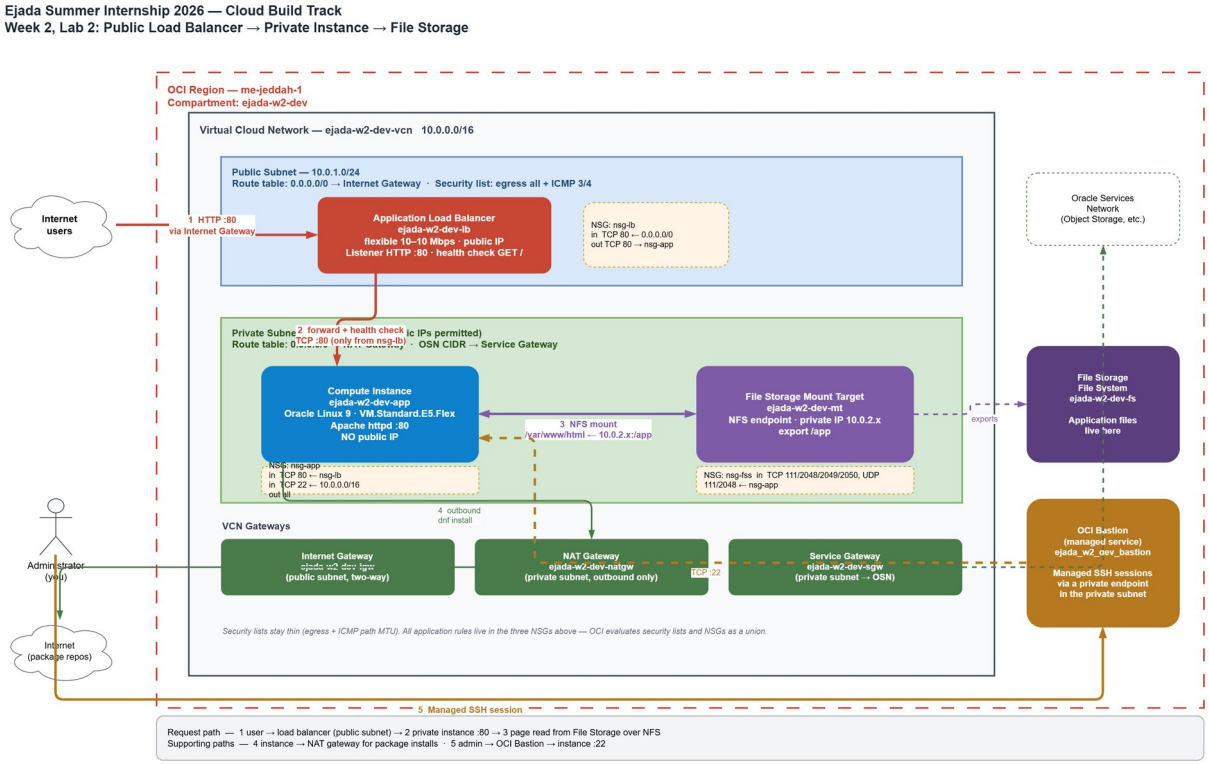

Figure 8 — Consolidated architecture: public subnet with the Application Load Balancer, private subnet with the compute instance and File Storage mount target, the three VCN gateways, and the numbered request and support paths.

Traffic paths

#	Path
1	Internet client → Application Load Balancer, HTTP :80, arriving via the Internet Gateway
2	Load balancer → compute instance :80 — forwarded requests and health checks, permitted only from the load balancer tier
3	Compute instance ↔ File Storage mount target — NFS over TCP 111 and 2048–2050, UDP 111 and 2048
4	Compute instance → NAT Gateway → internet — outbound only, for package installation and agent traffic
5	Private subnet → Service Gateway → Oracle Services Network — OCI service traffic kept on the Oracle backbone

No path exists from the internet to the private subnet. Path 4 is outbound-only by construction, because a NAT Gateway has no mechanism for accepting inbound connections, and path 2 is the sole route by which external traffic reaches the application.

12. Terraform Implementation (Step 2)

The environment described in this report is also expressed entirely in Terraform. The configuration is split across nine files by concern rather than collected in a single main.tf, and contains no hardcoded values: every OCID, CIDR, port, shape and name is supplied by a variable or derived in locals.tf.

File layout

File	Contents
provider.tf	Provider and version constraints
variables.tf	Every input — typed, described, several with validation blocks
locals.tf	Derived values: CIDRs, resource names, NFS port lists, common tags
data.tf	Lookups: availability domains, platform image, OSN service, mount target IP
network.tf	VCN, three gateways, two route tables, two security lists, two subnets
security.tf	Three NSGs and their rules
storage.tf	File system, mount target, export
compute.tf	Private instance and its cloud-init bootstrap
loadbalancer.tf	Load balancer, backend set, backend, listener
bastion.tf	OCI Bastion — gated behind a variable, disabled by default
outputs.tf	Application URL, addresses, mount command, bastion command

Design decisions in the code

Requirement	How it is met
No static values	Subnet CIDRs derived with cidrsubnet() from a single vcn_cidr variable; the mount target IP resolved by a data source; the OS image looked up rather than pinned by OCID
Variables and locals	30+ typed variables with descriptions; locals.tf derives names, CIDRs, the availability domain, NFS port lists and a shared tag map
Multiple .tf files	Nine files split by concern
.tfvars for values	terraform.tfvars (git-ignored) plus a committed terraform.tfvars.example
Data sources	Availability domains, platform image, Oracle Services Network, mount target private IP
Dependencies	Implicit through references, plus an explicit depends_on so the instance never boots before its NFS export exists
Lifecycle rules	ignore_changes on the instance image so a newer platform image does not silently rebuild the instance

Validation result

The configuration was formatted, initialised, validated and planned against the live oracle/oci provider (v8.26.0).

```
$ terraform validate
Success! The configuration is valid.

$ terraform plan -out=tfplan
data.oci_core_services.oracle_services_network: Read complete
```

```
data.oci_identity_availability_domains.this: Read complete
data.oci_core_images.app: Read complete
...
Plan: 33 to add, 0 to change, 0 to destroy.
```

```
PS E:\Ejada\Week 2\terraform> terraform validate
Success! The configuration is valid.

PS E:\Ejada\Week 2\terraform> terraform plan -out=tfplan
data.oci_core_services.oracle_services_network: Reading...
data.oci_identity_availability_domains.this: Reading...
data.oci_core_images.app: Reading...
data.oci_core_services.oracle_services_network: Read complete after 0s [id=CoreServicesDataSource-0]
data.oci_identity_availability_domains.this: Read complete after 0s [id=IdentityAvailabilityDomainsDataSource-2967461625
]
data.oci_core_images.app: Read complete after 0s [id=CoreImagesDataSource-1121111140]
```

Figure 9 — terraform validate succeeding, and terraform plan resolving all three data sources against the live OCI API.

The plan resolves every data source against the live API: the image lookup returns Oracle-Linux-9.8-2026.07.20-1 — the same build used in the manual deployment — and the Oracle Services Network filter resolves to a real service OCID. The private subnet plans with prohibit_public_ip_on_vnic set to true and the instance with assign_public_ip set to false, so the central guarantee of the design is expressed in code rather than relied upon at launch time.

Note bastion.tf is gated behind the create_bastion variable, which defaults to false. The plan therefore contains no bastion resources and terraform apply succeeds without the manage bastion-family grant discussed in Section 9, Issue 4. Setting the variable to true once the policy is in place adds the bastion without any other change to the configuration.

13. Technical Summary

Component	Configuration
Region / Compartment	me-jeddah-1 / intern-04-andrew-hany-cmp
VCN	ejada-w2-dev-vcn — 10.0.0.0/16
Public subnet	ejada-w2-dev-subnet-public — 10.0.1.0/24, Regional
Private subnet	ejada-w2-dev-subnet-private — 10.0.2.0/24, Regional
Gateways	Internet, NAT and Service gateways, all Available
Security lists	Two thin lists — egress all plus ICMP type 3 code 4
Network Security Groups	nsg-lb, nsg-app, nsg-fss — NSG-to-NSG rules
File Storage	ejada-w2-dev-fs, mount target 10.0.2.177, export /app
Compute instance	ejada-w2-dev-app — Oracle Linux 9, VM.Standard.E5.Flex
Instance addressing	Private 10.0.2.185, no public IP address
Application	Apache httpd on port 80, DocumentRoot on the NFS mount
Load balancer	ejada-w2-dev-lb — public, flexible 10/10 Mbps, HTTP :80
Administrative access	cloud-init at first boot (bastion unavailable — Issue 4)
Provisioning method	OCI Console and OCI CLI; Terraform in Step 2

14. Conclusion

This lab delivers a working two-tier architecture in which the application server is genuinely unreachable from the internet. The separation is enforced at three independent layers rather than asserted once: the

subnet prohibits public IP addresses on any VNIC within it, the route table offers only a NAT path outward with no inbound equivalent, and the Network Security Group admits traffic on port 80 exclusively from the load balancer's own security group rather than from an address range.

The storage design follows the same principle. Application files live on an OCI File Storage export mounted at the Apache document root, so the instance holds no application state and can be destroyed and rebuilt without data loss — which is exactly what happened during this deployment when the bootstrap approach had to change.

The IAM constraints encountered in Section 9 changed the implementation but not the outcome. With neither the Bastion service nor Run Command available, the application was installed through cloud-init supplied at launch. That path required no inbound administrative access whatsoever, and produced a configuration that is reproducible by definition — the same script is used by the Terraform implementation in Step 2, where the entire environment described in this report is expressed as code.

— End of Report —